Manual Técnico del Lenguaje OakLand

Índice

1. Introducción
2. Gramatical del Lenguaje OakLand
    - Declaraciones
    - Variables
    - Tipos de datos
    - Operadores
3. Clases Principales
    - Entorno
    - Visitantes (Visitor Pattern)
    - Clase: Bloque
    - Clase: If
    - Clase: While
4. Archivos del Proyecto
    - analizador.pegjs
    - clase.js
    - config.js
    - entorno.js
    - instancia.js
    - interprete.js
    - tool.js
    - visitor.js
5. Funciones Embebidas

1. Introducción

Este manual técnico abarca los detalles de implementación del lenguaje OakLand, un lenguaje diseñado para la manipulación de estructuras de datos, funciones embebidas y control de flujo basado en clases JavaScript. Se detalla el análisis de código, los elementos gramaticales, la ejecución de instrucciones y la representación de entornos de ejecución.

2. Gramatical del Lenguaje OakLand

El lenguaje OakLand permite la declaración y manipulación de variables, matrices, estructuras (structs) y tipos de datos primitivos. La estructura gramatical sigue una serie de reglas definidas por el archivo principal analizador.pegjs.

Declaraciones

Las declaraciones permiten la creación de variables, funciones y estructuras. Algunas de las declaraciones más relevantes incluyen:

```
Declaracion = dcl:Class _ { return dcl }

        / dcl:VarDcl _ { return dcl }

        / dcl:Funcion _ { return dcl }

        / stmt:Stmt _ { return stmt }
```

Variables

La declaración de variables permite la creación de variables de distintos tipos, incluyendo tipos primitivos y estructuras complejas como arrays y matrices.

```
VarDcl = tipo:Tipo _ id:Identificador _ "=" _ exp:Expresion _ ";"{ return crearNodo('declaracionVariable', { id, exp, tipo }) }
```

Tipos de Datos

Se manejan varios tipos de datos dentro de OakLand, que incluyen enteros, flotantes, cadenas y arrays. También se soportan estructuras (structs).

Operadores

OakLand soporta una variedad de operadores aritméticos, lógicos y de comparación que permiten realizar operaciones sobre los datos declarados.

3. Clases Principales

Entorno

La clase Entorno es esencial para la gestión de variables y símbolos en OakLand. Cada entorno representa un ámbito de ejecución y permite almacenar variables y su valor asociado.

```
export class Entorno {

    constructor(padre = undefined) {

        this.valores = {};

        this.padre = padre;

        this.simbolos = {};

    }

    setVariable(nombre, { tipo, valor }, linea = 0, columna = 0) {

        if (this.existeEnEntornoActual(nombre)) {

            throw new Error(`Variable ${nombre} ya está definida en este entorno`);

        }

        this.valores[nombre] = { tipo, valor };

        this.agregarSimbolo(nombre, tipo, valor, linea, columna);

    }

    getVariable(nombre) {

        const variable = this.valores[nombre];

        if (variable !== undefined) {

            return variable;

        }

        if (this.padre) {

            return this.padre.getVariable(nombre);

        }

        throw new Error(`Variable ${nombre} no definida`);

    }}
```

Esta clase se encarga de:

- **Almacenar variables:** mediante setVariable().
- **Recuperar valores de variables:** a través de getVariable().
- **Mantener símbolos:** con agregarSimbolo().

Visitantes (Visitor Pattern)

El patrón de visitante se utiliza para recorrer las distintas expresiones del lenguaje. Cada tipo de nodo tiene su propia implementación de visitante, lo que permite la evaluación dinámica del código.

Algunas funciones destacadas incluyen:

```
visitParseInt(node) {
    const valor = node.exp.accept(this);
    if (typeof valor === 'string') {
        return parseInt(valor);
    } else {
        throw new Error("parseInt solo acepta cadenas");
    }
}
```

4. Archivos del Proyecto

analizador.pegjs

El archivo analizador.pegjs contiene la gramática que define la estructura del lenguaje OakLand. Aquí se especifican las reglas para el análisis de código y la creación de nodos sintácticos.

clase.js

Este archivo contiene la definición de las clases principales que se utilizan en el lenguaje.

config.js

Archivo de configuración que almacena los parámetros globales que se utilizan en el lenguaje OakLand.

entorno.js

Implementación de la clase Entorno, que permite el manejo del entorno de ejecución.

instancia.js

Aquí se definen las instancias de las clases, así como la creación y manejo de objetos.

interprete.js

Este archivo contiene el intérprete principal del lenguaje, donde se manejan la ejecución de las sentencias y expresiones.

tool.js

Este archivo provee herramientas adicionales que facilitan el uso y manipulación del código, como utilidades para manejar datos y estructuras.

visitor.js

Este archivo contiene las implementaciones de las funciones visitantes, que recorren y ejecutan las distintas expresiones del código OakLand.

5. Funciones Embebidas

OakLand incluye varias funciones embebidas para la manipulación de datos y cadenas. Algunas de estas funciones son:

- parseInt(): Convierte una cadena en un número entero.
- parseFloat(): Convierte una cadena en un número decimal.
- toLowerCase(): Convierte una cadena a minúsculas.
- toUpperCase(): Convierte una cadena a mayúsculas.

```
visitToLowerCase(node) {
   const valor = node.exp.accept(this);
   if (typeof valor === 'string') {
      return valor.toLowerCase();
   } else {
      throw new Error("toLowerCase solo funciona con cadenas");
   }
}
```